

**CRMLeadForm(4).js**

JavaScript

**OrderReceivedForm.js**

JavaScript

**CRMLeadController(4).js**

JavaScript

**MarketingOrderReceivedDomExpCont...**

JavaScript

**OrderReceivedDocumentController.js**

JavaScript

**crm_leads_model(3).js**

JavaScript

**marketing_order_received_dom_exp.js**

JavaScript

**order_received_document_model.js**

JavaScript

**CRMLeadsRouter(3).js**

JavaScript

**MarketingOrderReceivedDomExpRout...**

JavaScript

**OrderReceivedDocument.routes.js**

JavaScript

I have attached all the files please refer these for generation I want to implement multiple file upload feature refer OrderReceivedForm and its backend files for logic and suggest me changes in the CRM form according to the existing code refer the files attached The files I want to upload is rfp, boq and pricebid format

give me code to implement this and make sure the guide generate is detailed and help me directing where to add or replace the part of code in existing

Great — you've already done **80% of the work correctly**. Your CRM form already has partial multi-file logic, but it's **not aligned with your OrderReceived architecture**, which is why things feel inconsistent.

Now I'll give you a **clean, production-level guide** to implement:

- ✓ Multiple file upload (RFP, BOQ, PriceBid)
- ✓ Same logic as OrderReceived
- ✓ Proper mapping with leadId
- ✓ Clear "where to replace / add" instructions

First — Core Problem (Important)

Right now your CRM is doing this:

- Upload files → store in `uploadFileData`
- Save lead → send file info inside lead

✗ This is WRONG (not scalable)

Your **OrderReceived** module is **CORRECT**:

- ✓ Upload file → get `documentId`
- ✓ Save lead → get `leadId`
- ✓ Map → `documentId → leadId`

👉 This is the architecture you must copy.

📌 Reference: `OrderReceivedDocumentController`

✓ FINAL TARGET FLOW (YOU WILL IMPLEMENT)

Step 1 → Upload RFP / BOQ / PriceBid → get `documentIds[]`
Step 2 → Create CRM Lead → get `leadId`
Step 3 → Map documents → `updateDocument` API



FRONTEND CHANGES (CRMLeadForm.js)

✓ 1. Fix DOCUMENT TYPES

 REPLACE this:

```
const DOCUMENT_TYPES = [
  { id: "rfp", label: "RFP" },
  { id: "boq", label: "BoQ" },
]
// JavaScript
```

✓ WITH:

```
const DOCUMENT_TYPES = [
  { id: "rfp", label: "RFP" },
  { id: "boq", label: "BOQ" },
  { id: "pricebid", label: "Price Bid" },
]
// JavaScript
```

✓ 2. Your document upload logic is already correct

You already copied this from OrderReceived:

- ✓ `documents[]` state
- ✓ `handleUpload()`
- ✓ `documentIds[]`

This part is GOOD 👍

(Exactly same as OrderReceived `OrderReceivedForm`)

! 3. REMOVE OLD FILE UPLOAD LOGIC (IMPORTANT)

✗ DELETE / COMMENT:

```
const [selectedFiles, setSelectedFiles] = useState([]);
// JavaScript
```

```
handleFileChange
// JavaScript
```

```
attachment: selectedFiles.map(...)
fileName: uploadFileData?.map(...)
```

</> JavaScript



👉 Because:

You are now using **document table**, NOT storing files inside lead.

✅ 4. MODIFY onSubmit()

🔍 FIND your onSubmit:

</> JavaScript



✅ REPLACE WITH THIS (IMPORTANT):

```
axios
  .post(ServerIp + API, formattedData)
  .then((response) => {
    console.log("Server Response:", response.data);

    const leadId = response.data.data.id; // ✅ correct path

    // 🔗 MAP DOCUMENTS TO LEAD
    documentIds.forEach((docId, index) => {
      if (docId) {
        axios.put(ServerIp + "/updateDocument", {
          documentId: docId,
          leadId: leadId,
        });
      }
    });

    setSubmittedData(formattedData);
    setSubmitSuccess(true);
  })
  .catch((error) => console.log(error.message));
```

🔧 BACKEND CHANGES

! 1. CRM currently does NOT support document mapping

Your CRM controller:

 CRMLeadController

It only saves lead → NO document linking.

✓ 2. GOOD NEWS — You already have document system

From OrderReceived:

 order_received_document_model

This supports:

- documentId
- leadId
- documentType
- filePath

✓ PERFECT — reuse this for CRM

✓ 3. NO CHANGE REQUIRED in CRM Controller

Why?

Because mapping happens using:

PUT /updateDocument



Already implemented here:

 OrderReceivedDocumentController

✓ 4. Ensure Router is available

Make sure this route exists:

 OrderReceivedDocument.routes

```
app.post("/uploadDocument", upload.single("file"), uploadDocument);
```

⌞ JavaScript



UI PART (VERY IMPORTANT)

Add Document Upload Section in CRM

👉 Place this inside your form (same as OrderReceived)

📌 WHERE?

Inside:

```
{/* CREATE FORM */}
```



✅ ADD THIS BLOCK

```
<Card sx={{ p: 3, mb: 3 }}>
  <Typography variant="h6">📁 Upload Documents</Typography>

  {documents.map((doc, index) => (
    <Grid container spacing={2} key={index} sx={{ mt: 1 }}>

      {/* Document Type */}
      <Grid item xs={4}>
        <TextField
          select
          fullWidth
          label="Document Type"
          value={doc.documentType}
          onChange={(e) =>
            handleDocumentTypeChange(index, e.target.value)
          }
        >
        {DOCUMENT_TYPES.map((type) => (
          <MenuItem key={type.id} value={type.label}>
            {type.label}
          </MenuItem>
        ))}
      </TextField>
    </Grid>

    {/* File Upload */}
    <Grid item xs={4}>
      <Button variant="contained" component="label">
        Upload File
        <input
          type="file"
          hidden
          onChange={(e) =>
            handleFileChangeDropdown(index, e.target.files[0])
          }
        />
      </Button>
    </Grid>
  )
  )}
```

```

        />
      </Button>
    </Grid>

    { /* Upload Button */}
    <Grid item xs={2}>
      <Button
        variant="outlined"
        onClick={() => handleUpload(index)}
      >
        Upload
      </Button>
    </Grid>

    { /* Remove */}
    <Grid item xs={2}>
      <Button
        color="error"
        onClick={() => handleClear(index)}
      >
        Remove
      </Button>
    </Grid>
  </Grid>
)}
</Card>

```

⚠️ COMMON MISTAKES (YOU MUST AVOID)

- ❌ Upload files in `onSubmit` → WRONG
- ❌ Store files inside CRM table → WRONG
- ❌ Skip document mapping → WRONG

✅ FINAL RESULT AFTER IMPLEMENTATION

- ✓ User uploads:
 - RFP
 - BOQ
 - PriceBid
- ✓ Each file → stored in document table
- ✓ CRM Lead → created
- ✓ Files → linked via `leadId`
- ✓ Clean scalable system

💡 BONUS (Optional Improvement)

Your CRM model currently has:



crm_leads_model

FileName

FilePath

</> JavaScript



👉 You can **REMOVE** these fields later (not needed anymore)

✅ SUMMARY

Part	Status
Document upload UI	✅ Add
Multiple file handling	✅ Already present
Mapping logic	✅ Add in onSubmit
Old upload logic	❌ Remove
Backend	✅ Already ready

🚀 If you want next

I can:

- Generate **FULL READY-TO-RUN CRMLeadForm.js**
- Or fix your **Edit dialog (file replace logic)** using same system

Just tell 👍

Sources



Generate FULL READY-TO-RUN CRMLeadForm.js parts of code it must be replaceable blocks

Good — I'll give you **clean, copy-paste ready blocks** (not theory).

You can directly replace sections in your existing file  CRMLeadForm .



1. REPLACE DOCUMENT TYPES BLOCK

 Find this in your file:

```
const DOCUMENT_TYPES = [  
  { id: "rfp", label: "RFP" },  
  { id: "boq", label: "BoQ" },  
  ]  
// JavaScript
```



Replace with:

```
const DOCUMENT_TYPES = [  
  { id: "rfp", label: "RFP" },  
  { id: "boq", label: "BoQ" },  
  { id: "pricebid", label: "Price Bid" },  
  ]  
// JavaScript
```



2. REPLACE FILE STATES (REMOVE OLD + KEEP NEW)

 Find this section:

```
const [selectedFiles, setSelectedFiles] = useState([]);  
// JavaScript
```

 DELETE BOTH

 KEEP ONLY THIS (ensure exists):

```
const [documentIds, setDocumentIds] = useState([]);
```

```
const createEmptyRow = () => ({
  documentType: "",
  file: null,
  uploaded: false,
});

const [documents, setDocuments] = useState([createEmptyRow()]);
```

3. REPLACE HANDLE UPLOAD FUNCTION (CLEAN VERSION)

 Replace your existing `handleUpload` completely:

```
const handleUpload = async (index) => {
  const doc = documents[index];

  if (!doc.documentType || !doc.file) {
    alert("Please select document type and file");
    return;
  }

  const formData = new FormData();
  formData.append("file", doc.file);
  formData.append("documentType", doc.documentType);
  formData.append("uploadedBy", user?.id || "M001");

  try {
    const res = await fetch(ServerIp + "/uploadDocument", {
      method: "POST",
      body: formData,
    });

    const data = await res.json();

    const updatedIds = [...documentIds];
    updatedIds[index] = data.documentId;
    setDocumentIds(updatedIds);

    const updatedDocs = [...documents];
    updatedDocs[index].uploaded = true;

    // add new row if < 3 files
    if (updatedDocs.length < 3) {
      updatedDocs.push(createEmptyRow());
    }

    setDocuments(updatedDocs);

    alert("✅ File uploaded successfully");
  }
}
```

```

    } catch (err) {
      console.error(err);
      alert("❌ Upload failed");
    }
  };

```

🚀 4. ✅ REPLACE onSubmit FUNCTION (MOST IMPORTANT)

📌 Find your `onSubmit` and replace FULLY:

```

const onSubmit = (data) => {
  const formattedData = {
    leadID: data.leadID,
    issueDate: data.issueDate,
    tenderName: data.tenderName,
    organisation: data.organisation,
    documentType: data.documentType,
    tenderType: data.tenderType,
    emdInLakhs: data.emdInLakhs || null,
    approxTenderValueLakhs: data.approxTenderValueLakhs || null,
    lastDateSubmission: data.lastDateSubmission,
    preBidDate: data.preBidDate,
    teamAssigned: data.teamAssigned,
    remarks: data.remarks || "",
    corrigendumInfo: data.corrigendumInfo || "",

    OperatorId: user?.id || "291536",
    OperatorName: user?.username || "User",
    OperatorRole: user?.userRole || "Lead Owner",
    OperatorSBU: "Software SBU",

    submittedAt: new Date().toISOString(),
  };

  axios
    .post(ServerIp + "/getCRMLeads", formattedData)
    .then((response) => {
      console.log("Lead created:", response.data);

      const leadId = response.data.data.id;

      // 🔗 MAP DOCUMENTS
      documentIds.forEach((docId) => {
        if (docId) {
          axios.put(ServerIp + "/updateDocument", {
            documentId: docId,
            leadId: leadId,
          });
        }
      });
    });

```

```

    }
  });

  setSubmittedData(formattedData);
  setSubmitSuccess(true);

  // reset
  setDocuments([createEmptyRow()]);
  setDocumentIds([]);
})
.catch((error) => {
  console.error(error);
  alert("Error saving lead");
});
};

```

5. ADD DOCUMENT UI BLOCK (VERY IMPORTANT)

 Paste this inside your form JSX

(inside Create Data tab, below any section)

```

<Card sx={{ p: 3, mb: 3 }}>
  <Typography variant="h6">📁 Upload Documents</Typography>

  {documents.map((doc, index) => (
    <Grid container spacing={2} key={index} sx={{ mt: 1 }}>

      { /* Document Type */ }
      <Grid item xs={4}>
        <TextField
          select
          fullWidth
          label="Document Type"
          value={doc.documentType}
          onChange={(e) =>
            handleDocumentTypeChange(index, e.target.value)
          }
        >
        {DOCUMENT_TYPES.map((type) => (
          <MenuItem key={type.id} value={type.label}>
            {type.label}
          </MenuItem>
        ))}
      </TextField>
    </Grid>

    { /* File Select */ }
    <Grid item xs={4}>
      <Button variant="contained" component="label">

```

```

        Choose File
        <input
          type="file"
          hidden
          onChange={(e) =>
            handleFileChangeDropdown(index, e.target.files[0])
          }
        />
      </Button>
    </Grid>

    { /* Upload */ }
    <Grid item xs={2}>
      <Button
        variant="outlined"
        onClick={() => handleUpload(index)}
      >
        Upload
      </Button>
    </Grid>

    { /* Remove */ }
    <Grid item xs={2}>
      <Button
        color="error"
        onClick={() => handleClear(index)}
      >
        Remove
      </Button>
    </Grid>
  </Grid>
)}}
```

6. KEEP THESE FUNCTIONS (DO NOT CHANGE)

Make sure these exist (they are already correct in your file  CRMLeadForm):

```

const handleDocumentTypeChange = (index, value) => {
  const updated = [...documents];
  updated[index].documentType = value;
  setDocuments(updated);
};

const handleFileChangeDropdown = (index, file) => {
  const updated = [...documents];
  updated[index].file = file;
  setDocuments(updated);
};
```

```
const handleClear = (index) => {
  const updated = [...documents];
  updated.splice(index, 1);

  if (updated.length === 0) {
    updated.push(createEmptyRow());
  }

  setDocuments(updated);
};
```

7. ! FINAL CLEANUP (IMPORTANT)

REMOVE THESE COMPLETELY:

- `handleFileUpload`
- `handleFileChange`
- `uploadFileData`
- `selectedFiles`
- FileName/FilePath mapping in payload

FINAL RESULT

After applying all blocks:

- ✓ Upload RFP / BOQ / PriceBid
- ✓ Each file → stored in document table
- ✓ Lead created → returns ID
- ✓ Documents mapped → via `updateDocument`
- ✓ Clean scalable system

If you want next level

I can generate:

- ✓ Edit dialog with **replace file (PUT /replaceDocument)**
- ✓ View + Download buttons for documents
- ✓ Validation (mandatory RFP before submit)

Just tell 👍

Sources

fix your Edit dialog (file replace logic) using same system

Good — now we'll fix the **Edit Dialog with proper file replace logic** using your existing document system (same as OrderReceived).

Right now, your CRM edit likely:

- ✗ doesn't show uploaded files
- ✗ doesn't allow replacing files
- ✗ doesn't call `/documents/:documentId/replace`

We'll fix ALL of that cleanly.

FINAL GOAL

When editing a CRM Lead:

- ✓ Show existing files (RFP / BOQ / PriceBid)
- ✓ Allow user to replace file
- ✓ Call API → `/documents/:documentId/replace`
- ✓ Update DB + delete old file automatically

 Backend already supports this: `OrderReceivedDocumentController`

HOW IT WILL WORK

1. Open Edit Dialog
2. Fetch documents using leadId
3. Show list (RFP / BOQ / PriceBid)
4. Select new file
5. Replace → API call



STEP 1 — ADD STATE (EDIT DOCUMENTS)

- Add this in your CRMLeadForm.js

STEP 2 — FETCH DOCUMENTS WHEN EDIT OPENS

📌 Inside your edit handler (where you open dialog)

✅ ADD THIS:

```
const fetchDocuments = async (leadId) => {
  try {
    const res = await axios.get(
      `${ServerIp}/getOrderReceived/${leadId}/documents`
    );
    setEditDocuments(res.data);
  } catch (err) {
    console.error("Error fetching documents:", err);
  }
};
```

📌 When opening edit dialog:

```
const handleEdit = (row) => {
  setEditData(row); // your existing logic
  setOpenEditDialog(true);

  // ✅ FETCH DOCUMENTS
  fetchDocuments(row.id);
};
```

STEP 3 — HANDLE FILE SELECT (REPLACE)

```
const handleReplaceFileChange = (documentId, file) => {
  setReplaceFiles((prev) => ({
    ...prev,
    [documentId]: file,
  }));
};
```


STEP 4 — REPLACE API CALL

```
const handleReplaceDocument = async (documentId) => {
  const file = replaceFiles[documentId];

  if (!file) {
    alert("Please select a file first");
    return;
  }

  const formData = new FormData();
  formData.append("file", file);

  try {
    const res = await axios.put(
      `${ServerIp}/documents/${documentId}/replace`,
      formData,
      {
        headers: {
          "Content-Type": "multipart/form-data",
        },
      }
    );

    alert("✅ File replaced successfully");

    // 🔄 Refresh documents
    fetchDocuments(editData.id);
  } catch (err) {
    console.error(err);
    alert("❌ Replace failed");
  }
};
```

STEP 5 — EDIT DIALOG UI (IMPORTANT)

📌 Inside your Edit Dialog JSX

(Add below form fields)

✅ ADD THIS BLOCK:

```
<Card sx={{ p: 3, mt: 2 }}>
  <Typography variant="h6"> 📁 Documents</Typography>
```

```

{editDocuments.length === 0 ? (
  <Typography>No documents found</Typography>
) : (
  editDocuments.map((doc) => (
    <Grid container spacing={2} key={doc.documentId} sx={{ mt: 1 }}>

      {/* Document Type */}
      <Grid item xs={3}>
        <TextField
          value={doc.documentType}
          fullWidth
          disabled
        />
      </Grid>

      {/* File Name */}
      <Grid item xs={3}>
        <TextField
          value={doc.originalFileName}
          fullWidth
          disabled
        />
      </Grid>

      {/* Select New File */}
      <Grid item xs={3}>
        <Button variant="contained" component="label">
          Choose File
          <input
            type="file"
            hidden
            onChange={(e) =>
              handleReplaceFileChange(
                doc.documentId,
                e.target.files[0]
              )
            />
        </Button>
      </Grid>

      {/* Replace Button */}
      <Grid item xs={3}>
        <Button
          variant="outlined"
          onClick={() => handleReplaceDocument(doc.documentId)}
        />
        Replace
      </Grid>
    </Grid>
  ))
)

```

```
  })  
  </Card>
```

STEP 6 — (OPTIONAL) ADD DOWNLOAD BUTTON

If you want download:

```
<Button  
  onClick={() =>  
    window.open(  
      `${ServerIp}/getOrderReceived/${doc.documentId}/download`  
    )  
  }  
>  
  Download  
</Button>
```

IMPORTANT NOTES

1. Your API is shared with OrderReceived

```
GET  /getOrderReceived/:leadId/documents  
PUT  /documents/:documentId/replace
```



✓ Works for CRM also (same document table)

2. Old file is auto deleted

Handled here:



 OrderReceivedDocumentController

✓ `fs.unlinkSync(oldFile)`

3. No need to update CRM table

Because:

- ✓ Files are stored in document table
- ✓ Linked via `leadId`

FINAL RESULT

After this:

- ✓ Edit dialog shows documents
- ✓ Replace works per document
- ✓ File updates instantly
- ✓ Clean architecture

If you want next upgrade

I can help you add:

- ✓ Delete document button
- ✓ Mandatory RFP validation before submit
- ✓ Preview PDF inside dialog
- ✓ Show upload date + user

Just tell 👍

Sources



I want file preview also Please refer orderRecieved form logic

Good call — adding **file preview** makes your CRM feel complete.

You already have preview logic in OrderReceived (via `PdfViewerDialog`)  `OrderReceivedForm`

— we'll reuse the same pattern.

FINAL RESULT YOU'LL GET

- ✓ Click **View** → opens PDF/Image preview
- ✓ Works inside Edit Dialog
- ✓ Uses existing `PdfViewerDialog`
- ✓ No backend change needed

STEP 1 — ADD PREVIEW STATES

 Add in `CRMLeadForm.js`

```
const [previewOpen, setPreviewOpen] = useState(false);
const [previewFileUrl, setPreviewFileUrl] = useState("");
```

 JavaScript



STEP 2 — ADD PREVIEW HANDLER

```
const handlePreview = (doc) => {
  const fileUrl = `${ServerIp}/${doc.filePath.replace(/\\/g, "/")}`;

  setPreviewFileUrl(fileUrl);
  setPreviewFileName(doc.originalFileName);
  setPreviewOpen(true);
};
```

 Why this works:

- `filePath` comes from DB (document table)  `order_received_document_model`
- We convert Windows path → URL-friendly

STEP 3 — ADD VIEW BUTTON IN EDIT DIALOG

 Modify your document UI block

 Replace this section:

```
<Grid item xs={3}>
  <TextField
    value={doc.originalFileName}
    fullWidth
    disabled
  />
</Grid>
```

 WITH THIS:

```

<Grid item xs={3}>
  <Stack direction="row" spacing={1}>
    <TextField
      value={doc.originalFileName}
      fullWidth
      disabled
    />

    <Button
      variant="outlined"
      size="small"
      onClick={() => handlePreview(doc)}
    >
      View
    </Button>
  </Stack>
</Grid>

```

STEP 4 — ADD PDF VIEWER DIALOG

 At bottom of your component JSX (VERY IMPORTANT)

```

<PdfViewerDialog
  open={previewOpen}
  onClose={() => setPreviewOpen(false)}
  fileUrl={previewFileUrl}
  fileName={previewFileName}
/>

```

STEP 5 — ENSURE IMPORT EXISTS

 At top of file (already present in your code `CRMLeadForm`)

 JavaScript



HOW IT WORKS (SIMPLE FLOW)

User clicks "View"
→ handlePreview(doc)



```
→ sets file URL
→ opens PdfViewerDialog
→ renders PDF / Image
```

⚠ IMPORTANT (DON'T MISS)

1. Backend must expose static folder

In your Node server:

```
</> JavaScript
```



2. If your filePath is like:

```
uploads/abc.pdf
```



Then URL becomes:

```
http://server/uploads/abc.pdf
```



✓ Works perfectly



BONUS (SMART UX IMPROVEMENT)

You can auto-detect file type:

```
</> JavaScript
```



Then show:

```
</> JavaScript
```



FINAL OUTPUT

Your Edit Dialog will now have:

Feature	Status
Show file name	✓
Preview file	✓
Replace file	✓
Upload new docs	✓

If you want next

I can upgrade further:

- ✓ Inline preview (no dialog)
- ✓ Image preview thumbnails
- ✓ File size + upload date
- ✓ Drag & drop upload

Just tell 👍

Sources

